Advanced Regular Expressions in Python

Pr. Zouhair ELAMRANI ABOU ELASSAD

Exercise 1: Server log analysis

A web server writes its events in a log file. Some lines are corrupted, some levels are written in lowercase, and some IP addresses are impossible (an octet greater than 255). You must read the file, parse every line with one verbose regular expression and produce a small report.

Data file: `exo1_server.log` **Level:** Advanced

Tasks:

1. Open `exo1_server.log` with a `with` block (encoding UTF-8) and load all its lines with `readlines()`.
2. Write ONE compiled pattern using `re.VERBOSE` (with comments) and named groups `date`, `time`, `level`, `ip`, `message`. The level must be matched case-insensitively. Lines that do not match must be collected in a list `bad_lines` together with their line number (starting at 1).
3. Normalise each level to uppercase and store each entry as a dictionary (use `groupdict()`).
4. Write a pattern `IPV4` that only accepts valid IPv4 addresses (each octet between 0 and 255, no leading zeros such as `01`) and use `re.fullmatch` to separate entries with a valid IP from entries with an invalid IP.
5. Using `collections.Counter`, count the number of `ERROR` entries per valid IP address and the number of entries per level.
6. From the messages of the ERROR entries, extract with ONE `re.findall` call all the identifiers written as `query_id=Q-4521` or `code=E-107` (key and value must be captured separately), then use `re.search` to compute the average duration of the database timeouts (`after 30s`).
7. Print a clean report: number of valid entries, bad lines, invalid IPs, errors per IP, extracted identifiers and the average timeout.

Expected output:

```
14 lines read
13 entries, 1 bad line(s): [(8, 'corrupted line without proper format')]
Invalid IPs: ['010.0.0.7', '256.1.1.1', '999.12.1.4']
Counter({'192.168.1.23': 2, '10.0.0.5': 2, '172.16.4.8': 1})
Counter({'ERROR': 6, 'INFO': 3, 'WARNING': 2, 'DEBUG': 1, 'CRITICAL': 1})
=== LOG REPORT ===
Parsed entries      : 13
Bad lines           : [8]
Entries invalid IP  : 3 -> ['010.0.0.7', '256.1.1.1', '999.12.1.4']
Errors per valid IP :
   192.168.1.23    2
   10.0.0.5        2
   172.16.4.8      1
Identifiers         : [('query_id', 'Q-4521'), ('query_id', 'Q-4533'), ('code', 'E-107'),
('query_id', 'Q-4602'), ('code', 'E-210'), ('code', 'E-300')]
Average timeout     : 29.0s over 3 timeouts
```

Exercise 2: Contact directory validation and normalisation

A directory of contacts was typed by hand. Emails and Moroccan phone numbers are written in many different ways and some are wrong. You must validate everything with regular expressions and produce a clean list.

Data file: `exo2_contacts.txt` **Level:** Advanced

Tasks:

1. Read the whole file with `f.read()` (in a `with` block) and also as a list of non-empty lines with `readlines()`.
2. On the whole text, use ONE `re.findall` with a **lookbehind** and a **lookahead** to extract all names (the text between `Name:` and the next `|`), without capturing the labels.
3. Write an email pattern validated with `re.fullmatch`: the local part (before @) is made of letters/digits separated by single `.`, `_` or `-` (no `..`, no leading/trailing separator); the domain contains at least one dot, labels may contain hyphens, and the final extension has at least 2 letters. Find which emails are rejected. (No leading/trailing separator means that the local part of the email must not start or end with ., _, or -).
4. Normalise phone numbers: first remove spaces, dots, dashes and parentheses with `re.sub`, then validate with `re.fullmatch` that the number is `+212`, `00212` or `0` followed by a mobile number starting with 6 or 7 and 9 digits in total. Convert every valid number to the format `+212XXXXXXXXX` (use a named group).
5. Build a list of valid contacts (valid email AND valid phone) as dictionaries, and print the invalid ones with the reason.
6. Use `re.sub` with a **function** to mask the emails of valid contacts: keep the first character of the local part and replace the rest by `***` (e.g. `a***@univ-cadi.ac.ma`).
7. Count the domains (use a lookbehind on `@`) of the valid emails with `Counter` and display the most common ones.

Expected output:

```
9 contacts
['Amine El Fassi', 'Salma Benali', 'John Smith', 'Yassine Idrissi', "Lina O'Brien", 'Bad Entry',
'Omar Tazi', 'Kenza Alaoui', 'Hamza Berrada']
VALID
   {'name': 'Amine El Fassi', 'email': 'amine.elfassi@univ-cadi.ac.ma', 'phone': '+212612345678'}
   {'name': 'Salma Benali', 'email': 'salma_benali@gmail.com', 'phone': '+212661234567'}
   {'name': 'Yassine Idrissi', 'email': 'y.idrissi@outlook.fr', 'phone': '+212661234567'}
   {'name': "Lina O'Brien", 'email': 'lina.obrien@my-startup.io', 'phone': '+212698765432'}
   {'name': 'Kenza Alaoui', 'email': 'kenza.alaoui@gmail.com', 'phone': '+212711223344'}
   {'name': 'Hamza Berrada', 'email': 'hamza@univ-cadi.ac.ma', 'phone': '+212700112233'}
REJECTED
   John Smith -> invalid email
   Bad Entry -> invalid email, invalid phone
   Omar Tazi -> invalid email
Amine El Fassi   a***@univ-cadi.ac.ma
Salma Benali     s***@gmail.com
Yassine Idrissi  y***@outlook.fr
Lina O'Brien     l***@my-startup.io
Kenza Alaoui     k***@gmail.com
Hamza Berrada    h***@univ-cadi.ac.ma
univ-cadi.ac.ma      2
gmail.com            2
outlook.fr           1
my-startup.io        1
```

Exercise 3: Cleaning and transforming a raw article

A report was exported from a web editor. It contains HTML comments (some on several lines), HTML tags, duplicated words, repeated spaces, dates in three different formats and amounts in two currencies. You must clean the text, standardise the dates, extract the amounts, and save the result in a new text file.

Data file: `exo3_article.html` **Level:** Advanced

Tasks:

1. Read the file `exo3_article.html` entirely with `read()`.
2. Remove all HTML comments, including the multi-line ones (hint: `re.DOTALL` and a **non-greedy** quantifier). Be careful not to delete everything between the first `<!--` and the last `-->`.
3. Remove the HTML tags, collapse repeated spaces/tabs into one space, and remove empty lines.
4. Remove duplicated consecutive words (`The the`, `was was`) with a **backreference**, ignoring case, and keep the first occurrence.
5. Convert every date to ISO format `YYYY-MM-DD` with ONE `re.sub` and a replacement **function**. The text contains `dd/mm/yyyy` dates, dates like `7 March 2016`, and dates that are already in ISO format (they must stay unchanged).
6. Extract all the amounts: `$1,299,450.75` (dollars) and `45,300.50 EUR` (euros). Use a pattern with named groups `usd` and `eur`, and lookarounds so that a number is never matched partially. Convert them to `float`.
7. Extract all the percentages (digits followed by `%`) using a lookahead, then write the cleaned text to `exo3_article_clean.txt` with `open(..., 'w', encoding='utf-8')`.

Expected output:

```
Annual Report 2025
The company was founded on 2015-11-03 and opened its first office in Casablanca on 2016-03-07.
In 2025, revenue reached $1,299,450.75, up from $980,200.00 in 2024. Operating costs were
45,300.50 EUR.
The next board meeting is scheduled for 2026-04-18 and the annual party is on 2026-12-21.
Customer satisfaction was very high, reaching 94% in Q4, compared to 88% in Q3.

Amounts    : [('USD', 1299450.75), ('USD', 980200.0), ('EUR', 45300.5)]
Percentages: [94, 88]
```

Exercise 4: From text blocks to CSV files

An online shop exports its orders as a text report. Each order is a multi-line block. You must extract the data with regular expressions and store it in two CSV files: one row per order and one row per ordered item.

Data file: `exo4_orders.txt` **Level:** Advanced

Tasks:

1. Read the whole file `exo4_orders.txt` with `read()`.
2. Write ONE pattern (`re.VERBOSE | re.MULTILINE`) that matches a complete order block and captures: `order_id`, `date`, `customer`, `email` (**optional**: a customer may have no email), `items` (the whole item section) and `status`. Use `re.finditer` and `groupdict()`.
3. For each order, apply a second pattern on the `items` section to extract `qty`, `product` and `unit_price` for each line like `- 2 x USB-C Cable @ 9.99`. Compute `line_total` and the order `total`.
4. Store the orders in `orders.csv` with the columns `order_id, date, customer, email, status, nb_items, total` using `csv.DictWriter` (open the file with `newline=""`). A missing email must be an empty cell.
5. Store the items in `order_items.csv` with the columns `order_id, qty, product, unit_price, line_total`. Note that one product name contains a double quote: check that the CSV is still correct.
6. Re-open `orders.csv` with `csv.DictReader` and compute (a) the number of orders per status and (b) the total revenue of all orders that are not `CANCELLED`.

Expected output:

```
4 orders found
order_id,date,customer,email,status,nb_items,total
1042,2026-02-03,Amine El Fassi,amine@example.com,SHIPPED,2,54.48
1043,2026-02-04,Salma Benali,salma.benali@mail.ma,DELIVERED,3,116.4
1044,2026-02-06,Guest User,,CANCELLED,1,36.25
1045,2026-02-09,Lina O'Brien,lina@my-startup.io,PENDING,2,339.0

order_id,qty,product,unit_price,line_total
1042,2,USB-C Cable,9.99,19.98
1042,1,Laptop Stand,34.5,34.5
1043,1,Mechanical Keyboard,89.0,89.0
1043,3,Notebook A5,2.5,7.5
1043,1,Desk Lamp,19.9,19.9
1044,5,HDMI Cable,7.25,36.25
1045,1,"27"" Monitor",249.0,249.0
1045,2,Webcam HD,45.0,90.0

Orders per status: {'SHIPPED': 1, 'DELIVERED': 1, 'CANCELLED': 1, 'PENDING': 1}
Revenue (without cancelled orders): 509.88
```